International Journal of Computers and Applications

ISSN: 1206-212X (Print) 1925-7074 (Online) Journal homepage: www.tandfonline.com/journals/tjca20

A new blendshape-based retargeting for 3D facial expression

Sol Lee, Hak-Bum Lee, Ye-Won Jang, Jung-Woo Kim & Young-Ho Seo

To cite this article: Sol Lee, Hak-Bum Lee, Ye-Won Jang, Jung-Woo Kim & Young-Ho Seo (2025) A new blendshape-based retargeting for 3D facial expression, International Journal of Computers and Applications, 47:12, 1007-1020, DOI: 10.1080/1206212X.2025.2571553

To link to this article: https://doi.org/10.1080/1206212X.2025.2571553

INTERNATIONAL JOURNAL OF COMPUTERS AND APPLICATIONS
2025, VOL. 47, NO. 12, 1007–1020
https://doi.org/10.1080/1206212X.2025.2571553

A new blendshape-based retargeting for 3D facial expression

Sol Lee [a], Hak-Bum Lee [b], Ye-Won Jang [a], Jung-Woo Kim [b] and Young-Ho Seo [c,d]

[a]R&D Team, Omotion Inc., Seoul, Republic of Korea; [b]Department of Electronic Materials Engineering, Kwangwoon University, Seoul, Republic of Korea; [c]Omotion Inc., Seoul, Republic of Korea; [d]Department of Materials Engineering, Kwangwoon University, Seoul, Republic of Korea

ABSTRACT

This paper proposes a deep-learning network that transfers human facial expressions to targets such as avatars. We utilize a graph convolutional autoencoder (GC AE) to extract features from human facial expressions and then convert these features into blendshape coefficients using a fully-connected (FC) layer. By estimating blendshape coefficients from a person's 3D facial data, we can animate avatars to replicate the same facial expressions as the person. In particular, we construct learning data features by combining deformation gradients and displacements (or deltas), and employ graph convolutional networks (GCN) to extract features from a person's 3D facial mesh effectively. We trained the network in a two-stage learning process. During the training, introducing masks for each blendshapes into the loss function resulted in significant learning effects. Visually, it was confirmed that the same expressions were estimated. Furthermore, we enhanced versatility through additional experiments using the FLAME (Faces Learned with an Articulated Model and Expressions) facial model, commonly employed in 2D image-based 3D face generation networks and voice-based 3D facial animation generation networks. This paper demonstrates the feasibility of accurately transferring 3D human facial expressions to other avatars or virtual humans.

ARTICLE HISTORY
Received 30 September 2024
Accepted 27 September 2025

KEYWORDS
Facial expression; deformation gradient; graph convolutional networks; blendshape; 3D facial expression

1. Introduction

Implementing avatar facial movements has been attempted in various ways in the movie [1], game, and virtual environment industries. In recent film productions, facial movements are primarily captured by marking an actor's face and tracking those markers to move the avatar's face. However, this method comes with the inconvenience of having to attach or mark the actor's face with markers. Moreover, in the virtual environment industry, there is a growing need for technological research to enhance the realism of digital humans. Consequently, recent research on avatar facial animation is being actively conducted from various perspectives [1], with particular focus on studies of facial expression transfer techniques based on deep learning [1]. Learning-based 3D facial expression creation methods have the advantage of reducing time and production costs and have enabled the creation of sophisticated animations thanks to recent advances in deep learning technologies. To provide clarity, it is important to distinguish between face parameterization methods and animation techniques. Parameterization methods define how a face is represented: for example, blendshapes encode expressions as vertex offsets from a neutral mesh, feature-point-based models assign weighted influences to landmarks, and neural face models capture high-dimensional expression features through latent spaces. By contrast, animation techniques such as rigging, deformation transfer, and motion retargeting describe how these representations are manipulated or transferred. Previous literature has sometimes blurred these categories, which can lead to conceptual confusion.

Traditional methods for animating faces include the rigging method [2] and the blendshape technique [3]. The rigging method is a technique where control points are inserted into a 3D model, and weights are assigned to each vertex influenced by these control points to manipulate facial expressions. These two methods are often used together to naturally represent the expressions of 3D characters[4]. Additionally, animating facial expressions using the deformation transfer method is also possible[5]. However, these traditional techniques are heavily labor-intensive, rely on expert design, and often lack scalability when applied to diverse avatars. This motivates the need for learning-based approaches that can generalize across subjects while reducing manual workload.

In recent times, alongside traditional methods, several studies have been conducted based on artifi-

CONTACT Young-Ho Seo yhseo@kw.ac.kr Kwangwoon University, Omotion Inc., Kwangwoon-ro 20, Nowon-gu, Seoul, Republic of Korea

cial intelligence. For facial feature extraction encoders, autoencoder, [6], variational autoencoders (VAE) [7] combined with the concept of graph convolution networks (GCNs) [8], such as graph convolutional AE (Graph AE) [9] and graph convolutional VAE (Graph VAE) [10, 11], are primarily used. Moreover, research [12] has been conducted using Diffusionnet [11] for feature extraction. Additionally, studies [13] have also been conducted on using 2D AE to generate avatar facial images from human facial footage for 3D retargeting. Neural Face Rigging (NFR) is a data-driven method that learns to animate and retarget 3D facial meshes using automatically estimated blendshape rigs. It enables realistic and controllable facial deformations directly from unrigged meshes in the wild, demonstrating strong generalization across diverse facial topologies [14]. Sparse2Dense is a framework for 3D object detection that transforms sparse point cloud features into dense representations through learned upsampling, improving localization and recognition performance in LiDAR-based scenes [15]. Noteworthy is the research that significantly enhances the realism and generalization performance of 3D facial expression animation by integrating landmark-based trajectory generation techniques with Sparse2Dense mesh reconstruction [16, 17]. While these learning-based approaches provide new opportunities, they are still limited in several ways: many require retraining for each new avatar, others fail to capture localized geometric details, and some rely on mesh reconstruction rather than coefficient prediction, which reduces compatibility with real-time pipelines.

However, current 3D facial animation methodologies have three main issues. First, not considering the three-dimensional information of the human face can reduce the accuracy and realism of facial expression extraction [13]. Especially when using only 2D information, there can be limitations in accurately representing the depth and volume of specific facial regions depending on the camera angle. Second, each avatar subject to animation requires independent learning [10]. This issue leads to a significant investment of time and effort in manually creating datasets that pair human input expressions with corresponding avatar expressions. Third, suppose there is no consideration for the range of movement between the input and output faces. In that case, the avatar's expression may be created too exaggerated or too subtle compared to the input, resulting in low-quality animation [5]. These limitations highlight the research problem we address: how to design a scalable learning-based system that can capture fine-grained expression features and retarget them across different avatars without retraining, while remaining compatible with commercial animation pipelines.

The proposed method directly predicts blendshape parameters (i.e. blendshape weights) instead of generating 3D meshes, thereby enabling seamless integration with widely used commercial graphics engines, such as Unreal Engine, Unity, and Blender. This output format adheres to standard facial animation pipelines, where predefined blendshape templates drive facial expressions, allowing our system to be immediately applicable in real-time production environments. Importantly, because the animation is parameterized through blendshape weights, it is inherently independent of the underlying mesh topology. As long as the 3D character is rigged with compatible blendshapes, our method can animate diverse characters with varying mesh resolutions or topologies. This decoupling from geometry makes the approach highly versatile, scalable, and practical for deployment in a wide range of applications, from virtual avatars to game characters and digital humans. While encoder–decoder architectures such as GCN AE or MLP AE are widely used in recent works [18, 19], our framework leverages a Graph VAE to enforce a smoother and more disentangled latent space via KL-divergence regularization. This design improves robustness against overfitting and enables better generalization across diverse facial topologies, which makes Graph VAE particularly well-suited for 3D facial motion retargeting. While predicting blendshape coefficients offers strong compatibility with existing pipelines, prior methods remain limited by avatar-specific retraining and insufficient handling of expression-relevant regions. To overcome these challenges, we propose a novel framework that combines DR features and displacement within a Graph VAE, with a mask-based loss to emphasize critical regions such as lips and eyes.

To address these issues, this paper proposes a method for transferring facial expressions to 3D avatars. This is achieved by estimating the blendshape coefficients of facial expression features. The network, as described in Section 3, consists of a facial expression feature extracting encoder and a blendshape coefficient estimation decoder. The facial expression feature extraction encoder extracts features of facial expressions, transforming a neutral face into three-dimensional representations. Concurrently, the blendshape coefficient estimation network estimates the blendshape coefficients that make up the expression from the previously extracted facial features. During training, introducing a mask for each blendshape in the loss function significantly enhances learning effectiveness. Additionally, using the blendshape method in avatar animation has the advantage of eliminating the need for individual learning for each avatar. With the estimated blendshape coefficient values, it's possible to animate all avatars that are set up

with blendshapes. Moreover, since blendshaped expressions are hand-crafted by experts, they can reflect natural facial movements without any artifacts. Unlike prior works that rely on patch-based or region-based weighting strategies r], our method applies masking directly at the blendshape level. This approach takes advantage of semantically meaningful expression units (e.g. jaw open, eye blink) that are already defined in standard pipelines, thereby avoiding manual mesh segmentation, improving training stability, and ensuring alignment with production-ready animation frameworks. The main contributions of this paper are summarized as follows:

- We introduce a hybrid feature representation combining deformation representation and displacement, providing a robust basis for extracting expression-related variations in 3D facial meshes.
- We propose a GCN-based VAE framework that directly predicts blendshape coefficients, enabling avatar-independent retargeting and eliminating the need for per-avatar retraining.
- We design a blendshape mask-based loss function that enhances training stability and accuracy by selectively focusing on expression-critical regions.

Through extensive experiments, including comparisons with Apple's Live Link Face and Google's MediaPipe, we demonstrate that the proposed method achieves precision comparable to industry standard systems while offering superior flexibility and reproducibility.

In this paper, we first discuss related topics in Section 2, and then propose a facial expression transfer method based on the research findings in Section 3. We present experimental results in Section 4 and conclude our paper in Section 5.

2. Related works

Facial expression retargeting has been studied from multiple perspectives, ranging from marker-based capture systems to modern learning-based methods. In this section, we provide an overview of representative approaches in the retargeting pipeline, rather than focusing only on individual modules. Specifically, we review (i) marker- and rigging-based retargeting approaches, (ii) blendshape-based methods, (iii) learning-based methods including mesh-to-mesh and image-to-mesh retargeting, and (iv) recent graph-based neural network models. We then highlight the research gap that motivates our proposed method.

2.1. Marker-based and rigging-based retargeting

Early retargeting methods relied on motion capture systems using facial markers or optical tracking devices. While these methods can produce high-quality results, they are expensive, time-consuming, and not suitable for scalable or real-time applications. Rigging-based techniques, where control points are embedded into the mesh and weights are manually assigned, allow animators to retarget facial movements, but require significant artistic expertize and manual effort [2]. These limitations motivated the development of more automated approaches.

2.2. Delta blendshape-based facial animation

3D facial animation techniques are typically categorized into rigging-based methods, 3D Morphable Model (3DMM)-based methods [20], and blendshape-based methods [3, 21]. Among them, blendshape-based approaches are widely used due to their intuitive control over facial expressions. In particular, the delta blendshape method, which represents facial expressions as offsets from a neutral shape, enables the generation of natural and real-time animations, and has been broadly adopted in game engines and metaverse environments.

Blendshape is a technique commonly used in 3D modeling and animation, involving blending various 'shapes' to create different expressions [21]. In terms of facial animation creation, as shown in Figure 1, expressions (shapes) like left eye blink, right eye blink, and jaw open can be blended to create the expression on the far right, with both eyes closed and mouth open. Each of these shapes has the same topology, allowing for the calculation of displacement (delta) values by subtracting the vertices of the neutral shape from each shape's vertices. This animation method, using these displacements, is known as the delta blendshape method [3].

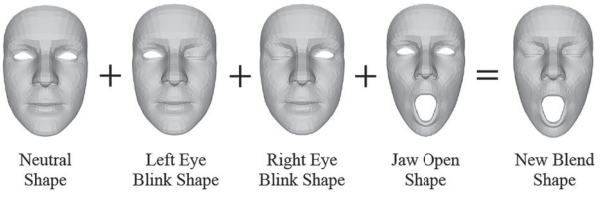

Figure 1. The concept of blendshape-based face animation. An image showing the process of creating a new blended facial expression by combining different facial shapes. From left to right: a neutral face shape, a left eye blink shape, a right eye blink shape, and a jaw open shape are added together to produce a final blended shape where the eyes are closed and the mouth is open. Each face is represented as a gray 3D model.

Typically, these delta values multiply a weight between 0 and 1 to determine the extent to which a particular expression is included. This is referred to as the blendshape coefficient or weight, where 0 means the expression is not included, and one means it is fully included. By continuously varying these values between 0 and 1, natural animations can be created. Occasionally, values greater than 1 are multiplied to create exaggerated expressions or negative values to create opposite movements.

Equation (1) represents facial animation using 52 delta blendshapes, where the delta, obtained by subtracting the neutral from each of the 51 shapes, is combined with the coefficient for each shape, and then the neutral shape is added back [22].

$$V_{\text{anim}} = V_{\text{neut}} + \sum_{i=1}^{51} (c_i (V_i - V_{\text{neut}})) = V_{\text{neut}} + \sum_{i=1}^{51} (c_i \phi_i) \quad (1)$$

While blendshapes offer controllability and compatibility with widely used animation pipelines such as ARKit, Unreal Engine, and ReadyPlayerMe, most existing learning-based methods focus on mesh reconstruction rather than directly predicting blendshape coefficients. This restricts their generalization, as they often require retraining for each avatar and are less reusable across different topologies.

2.3. Deformation representation feature and displacement

This paper uses feature learning to extract features from a person's 3D facial mesh. During this process, the learned features are composed by combining deformation representation (DR) features [10, 15, 23–25], which utilize the deformation gradient, with displacement. Figure 2 illustrates the schematic of the feature composition method.

Vertex displacement is commonly used as an input representation for feature learning on 3D meshes. However, this approach is sensitive to variations in scale and rotation of the face, which can limit its robustness and generalizability. To address these limitations, deformation representation (DR) features were proposed in prior works, and we adopt this representation in our study to improve training stability and generalization performance. The neutral mesh of a person is used as the reference mesh, and the mesh with expressions is considered the deformed mesh. The extent to which the vertices of the deformed mesh $V_{i,\text{gradient}}$ have changed from the reference mesh $V_{i,\text{ref}}$ can be represented by a $R^{3\times3}$ sized matrix, known as the deformation gradient.

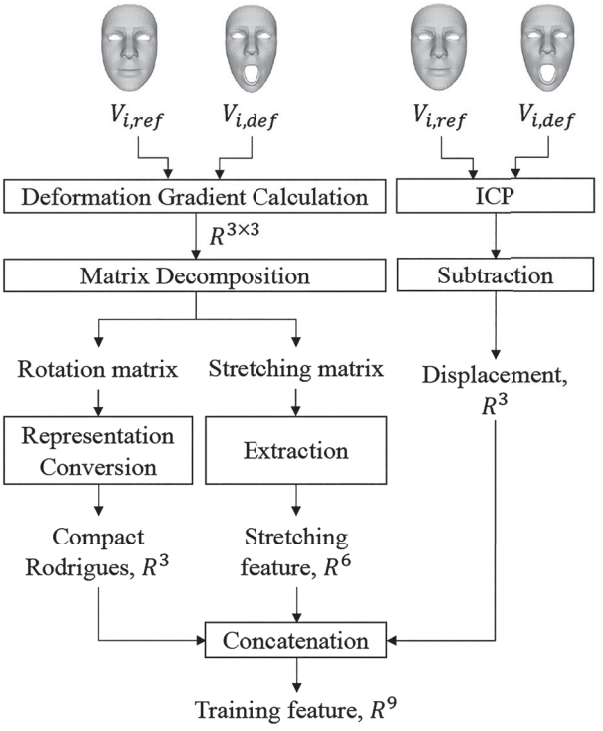

Figure 2. The process of calculating DR feature and displacement of the deform mesh from the reference mesh. A flowchart showing how 3D facial models (neutral and deformed) are processed to create a 9D training feature. The left path computes deformation gradients, decomposes them into rotation and stretching matrices, and converts them into a 3D Rodrigues vector and a 6D stretching feature. The right path uses ICP to compute 3D displacement. All features are concatenated into a 9D training vector.

This matrix can be decomposed through matrix decomposition into the product of a normal orthogonal matrix and a symmetric matrix. The normal orthogonal matrix, with its column and row vectors forming a normalized basis, has the property of preserving length, and its inverse matrix being the same as its transpose matrix, implies that the inverse represents a transformation in the opposite direction, allowing it to be viewed as a rotation matrix. The symmetric matrix, having all real eigenvalues and orthogonal eigenvectors, where each eigenvalue represents the degree of deformation and eigenvectors indicate the direction of deformation, can describe stretching or compression in space, hence it can be viewed as a stretching matrix. Therefore, by using preprocessing that decomposes the deformation gradient of the deformed mesh from the reference mesh into the rotation and stretching matrices, we aimed to effectively extract changes in expressions. Additionally, the displacement obtained by subtracting the reference mesh from the deformed mesh after applying ICP (Iterative Closest Point) was also included in the learning features [26].

The DR feature is a method for quantitatively describing local geometric changes between a reference 3D mesh

and its deformed version. It captures the deformation at each vertex by estimating a local linear transformation matrix that best explains the change in the relative positions of neighboring vertices. To make this representation suitable for machine learning models, the matrix $\mathbf{T}_i \in \mathbb{R}^{3\times3}$ is flattened into a 9-dimensional vector. This DR feature vector directly encodes the deformation of each vertex in a compact form and serves as an effective input for neural networks that aim to learn or predict mesh deformations. Although DR and displacement have been widely used in mesh reconstruction and shape analysis, their potential for direct retargeting into blendshape coefficients has not been fully explored. Our method leverages this combination to connect robust geometric encoding with practical coefficient-based animation pipelines.

2.4. Graph convolutional variational auto-encoder (Graph VAE)

Graph VAE is a neural network model designed to process graph data. This model combines the ideas of VAE with graph theory, extracting features of graph-structured data and learning low-dimensional representations of graph data.

Equation (2) is the propagation rule of GCN. GCN aggregates the features of connected nodes. Let's denote the data of the current layer as H^i and the data of the next layer as H^{i+1}. By multiplying H^i with the adjacency matrix A, it extracts the information of the connected nodes. To include the node itself, the identity matrix I is added to A and then multiplied with H^i, which allows for the extraction of information from the node itself and its connected nodes. This is multiplied by the weight matrix W and processed through the activation function $\sigma()$, conducting forward propagation.

$$H^{i+1} = \sigma\left((A+I)H^iW\right) \tag{2}$$

The goal of a typical VAE is to extract latent features of data and use them to generate new data similar to the original data. The goal of VAE can be represented by Equation (3). The left term inside min() is the reconstruction loss, composed of the data distribution between the input and reconstructed data. x is the original data, $q_\phi(z \mid x)$ is the distribution of the latent variable z defined by the encoder network, and $p_\theta(x \mid z)$ is the distribution of the data reconstructed by the decoder network. The right term is the KL (Kullback-Leibler divergence) loss [27]. It measures the difference between the distribution of the latent variable z generated by the encoder network $q_\phi(z \mid x)$ and a predefined target distribution (usually a standard normal distribution $p(z)$), ensuring that the latent space is formed in the desired direction as VAE learns.

$$\begin{aligned}\min L(\theta,\phi;x) = \min(&-\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] \\ &+ KL(q_\phi(z \mid x) \parallel p(z)))\end{aligned} \tag{3}$$

Facial meshes are naturally represented as graphs, where vertices act as nodes and edges preserve mesh connectivity. Graph VAE approaches have been successfully applied to 3D shape analysis and facial modeling. However, most of these works focus on mesh reconstruction and require avatar-specific retraining for retargeting tasks, which limits scalability. Our work addresses this by directly predicting coefficients in a topology-independent manner.

2.5. Discussion and research gap

To summarize, the retargeting literature spans marker-based, blendshape-based, and learning-based approaches. While prior studies demonstrated the usefulness of blendshapes, DR features, and Graph VAEs, three challenges remain unsolved:

- Heavy reliance on mesh reconstruction instead of coefficient prediction, which reduces pipeline compatibility.
- Avatar-specific retraining requirements, which hinder scalability and efficiency.
- Insufficient attention to expression-relevant regions, leading to unrealistic or unstable results.

Our work addresses these issues by directly predicting blendshape coefficients from DR and displacement features using a Graph VAE framework, and by introducing a blendshape mask-based loss to focus learning on critical facial regions. This results in a generalizable, scalable, and production-ready solution for real-time digital human animation.

3. Proposed retargeting method

3.1. Overview

Figure 3 represents a simplified illustration of the proposed facial expression transfer network. First, the encoder–decoder corresponds to the Graph VAE, which is used to extract features of expressions from a person's 3D facial mesh. In this case, the input to the encoder and the output of the decoder are not the mesh itself but the mesh features mentioned in Section 2.2. Next, the Fully-Connected (FC) layers, marked in green, correspond to the decoder that transforms the compressed expression

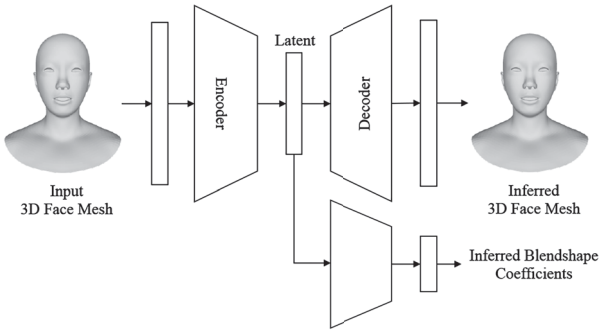

Figure 3. Proposed deep-learning-based facial expression retargeting method. A diagram illustrating a neural network pipeline that takes an input 3D face mesh and processes it through an encoder to generate a latent representation. This latent vector is then passed through a decoder to reconstruct the inferred 3D face mesh. Simultaneously, the latent representation is also used to predict inferred blendshape coefficients, capturing facial expression parameters.

features [28] (or latents) from the Graph VAE into blendshape coefficients. This network extracts features from the human facial mesh and predicts blendshape coefficients from these features. This decoder is designed to estimate the 52 blendshape coefficients defined by Apple ARKit. For training the network, we constructed a custom dataset consisting of Korean speech data from 10 native Korean speakers and newly created 52 blendshapes for each of these individuals [29]. In other words, our network is designed to extract features from a 3D facial mesh and predict 52 blendshape coefficients based on those features.

Compared with conventional GCN autoencoders (GCN AE) or multilayer perceptron autoencoders (MLP AE), our Graph VAE formulation has two distinct advantages. First, the KL-divergence regularization encourages a smoother and disentangled latent space, which reduces overfitting to specific mesh identities and supports better generalization across avatars. Second, the variational design allows stochastic sampling of latent variables, enabling richer expression variability and robustness to noise. These properties make Graph VAE particularly suitable for extracting 3D facial motion representations for retargeting [18, 19].

3.2. Network structure

The facial expression encoder extracts the latent features of expressions from the deformation features of a human 3D facial mesh as shown in the upper part of Figure 4. This network part consists of an encoder that extracts features from the human facial mesh and a decoder that restores the human facial mesh from these expression features. By updating the encoder and decoder with the error between the input features and the restored features, the network is trained to extract the characteristics of facial expressions effectively. The decoder that estimates blendshape coefficients from expression features. The middle part of Figure 4 shows the blendshape coefficient decoder. The features compressed by the expression encoder in the previous chapter are transformed into blendshape coefficients by a decoder consisting of fully connected layers.

The structure of the facial retargeting network is shown in Figure 4. This network consists of a facial encoder that takes features of shape (frame, number of vertices, 9) as input, and a decoder that generates a 51-dimensional blendshape coefficient vector. The network is composed of graph convolutional layers and fully connected layers.

3.3. Loss function

The proposed network combines a total of four loss functions. These are the feature reconstruction loss and KL loss for the expression extraction encoder, and the blendshape coefficient loss and vertex reconstruction loss for the blendshape coefficient decoder. To enhance the performance of retargeting, blendshape masks for each blendshape have been introduced. A blendshape mask is a mask that separates the moved and unmoved areas for each blendshape. The 3D face mesh includes not only the eyes and mouth, which are significantly involved in expressions, but also the back of the head, neck, and collarbone, which are less related to expressions. In reality, minor errors in these areas for each blendshape can collectively harm learning. To prevent this, vertices whose distance from the neutral is above a threshold were considered as involved in expressions, while those below the threshold were considered unrelated to expressions, and were masked with 1 and 0, respectively. An example visualized using these created masks can be seen in Figure 5.

Unlike patch-based or region-based weighting strategies [30], which divide the mesh spatially into local areas, our method applies masking directly at the semantic blendshape level. This design provides three advantages: (i) it leverages expression units (e.g. jaw open, lip corner raise) that are already meaningful in production pipelines, (ii) it avoids manual segmentation of mesh regions for each avatar, and (iii) it improves training stability by focusing losses on expression-critical regions such as lips and eyes.

The entire network is updated according to the loss formula in Equation (4). In the loss formula, l_f represents the feature reconstruction loss, l_{KL} is the KL loss, l_c is the blendshape coefficient loss, l_v is the vertex loss

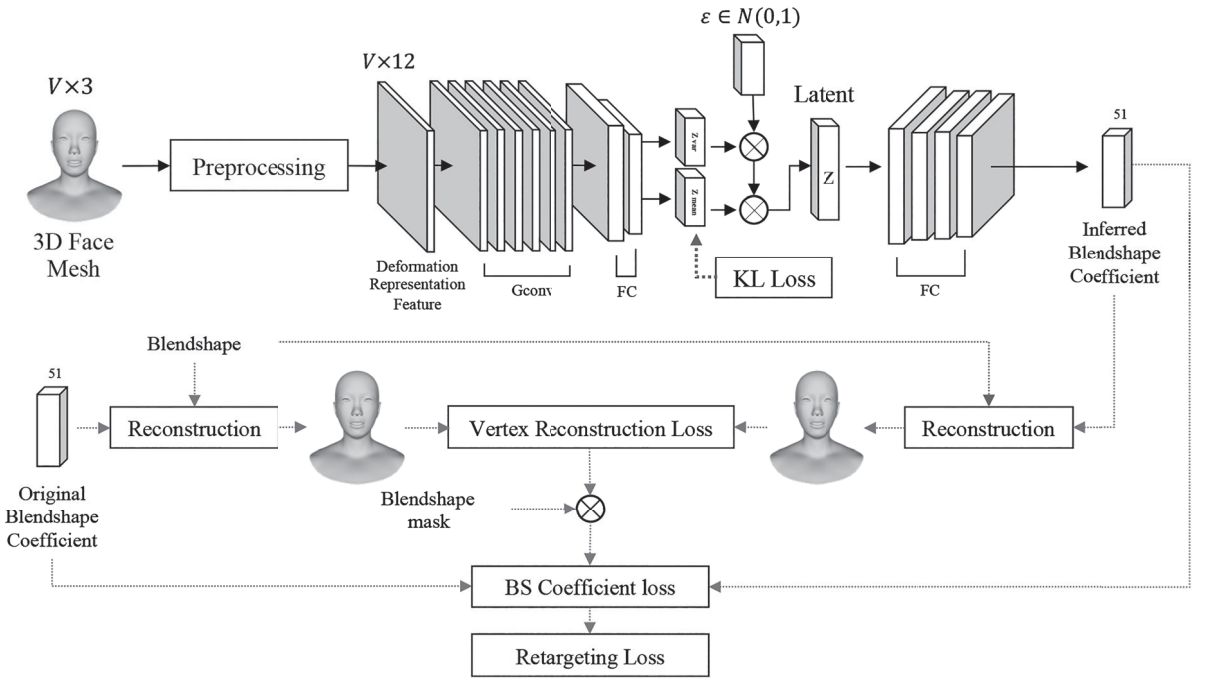

Figure 4. The network architecture of the facial expression retargeting network. A detailed diagram of a deep learning framework for 3D face mesh processing and blendshape coefficient inference. The top section illustrates a variational autoencoder (VAE) that encodes a 3D face mesh into a latent vector using graph convolutions (GConv) and fully connected layers (FC), incorporating both KL and reconstruction losses. The latent vector is decoded to reconstruct the 3D mesh and infer 51 blendshape coefficients. The bottom section compares the inferred coefficients with the original ones by reconstructing meshes and computing vertex reconstruction loss, blendshape coefficient loss, and retargeting loss using a blendshape mask.

reconstructed from blendshape coefficients, and M_b corresponds to the blendshape mask. Each loss is calculated using MSE (Mean Squared Error). The final term is the retargeting loss, with the formula as in Equation (5). Here, i is the blendshape index, c_i is the blendshape coefficient, and v_i is the reconstructed vertex.

$$l_{\text{total}} = \lambda_1 l_f + \lambda_2 l_{KL} + \lambda_3 l_c M_b l_v \quad (4)$$

$$\text{Retargeting Loss} = \lambda_3 \sum_{i=1}^{51} \left((c_{i,\text{label}} - c_{i,\text{pred}}) \times M_b (v_{i,\text{label}} - v_{i,\text{pred}}) \right) \quad (5)$$

4. Experimental results

In this section, we present a comprehensive set of experiments designed to validate the proposed facial expression retargeting method. The experiments are organized into four parts. First, we describe the implementation environment and the network configuration used for training and inference. Second, we report the process of hyper-parameter tuning and summarize the optimal settings. Third, we provide training results to illustrate the effectiveness of the loss design and the stability of the proposed network. Finally, we evaluate the performance of our method through both quantitative and qualitative comparisons. The quantitative evaluation focuses on vertex reconstruction accuracy against research baselines such as Sparse2Dense (S2D) and Neural Face Rigging (NFR), while the qualitative evaluation compares our method with widely used industry systems such as Apple's Live Link Face and Google's MediaPipe. Through this structure, we aim to demonstrate that our method is not only accurate and stable in retargeting facial expressions but also compatible with real-time production pipelines.

4.1. Environment

The network proposed in this paper was programed using Python and computations were carried out on a GPU using CUDA in a Linux environment. An NVIDIA RTX 4090 was used as the GPU, and an Intel (R) Core (TM) i9-10900X CPU @ 3.70 GHz was used as the CPU. It took 5 h to train for 100 epochs. The facial expression encoder is experimentally set with 7 Gconv layers and 2 layers of fully connected layers. The output dimensions of the graph convolution layers are each the number of

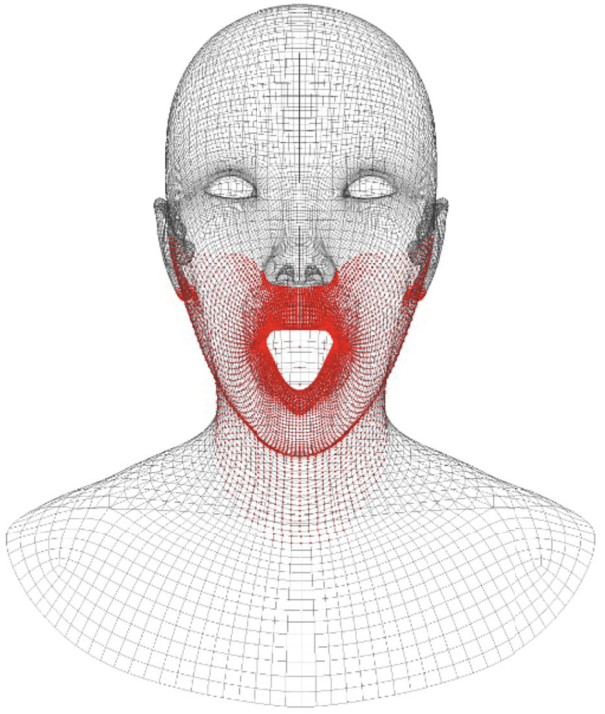

Figure 5. Blendshape mask of 'jawopen' shape visualizing vertices. A 3D wireframe model of a human head and upper torso with an open mouth expression. The mesh is mostly black, with red-highlighted vertices concentrated around the mouth, chin, and lower cheek area, indicating regions affected by a specific blendshape or deformation.

vertices multiplied by 256, 128, 128, 64, 32, 16, and 1, respectively. The output of the graph convolution layers thus becomes the number of vertices, which is then flattened and fed into the fully connected layers. The output dimensions of the fully connected layers are respectively twice the dimension of the latent space and the number of dimensions in the latent space. The decoder part of the facial expression encoder has a symmetric structure to the encoder.

The expression to blendshape coeff. decoder is experimentally set with 4 layers of fully connected layers. The output dimensions of the fully connected layers are set as 300, 300, 300, and 51, respectively, aligning the output dimensions with the number of blendshapes. For the layers with an output dimension of 300, ReLU is used as the activation function [31], and for the final layer, the tanh is employed [32].

For clarity, the complete architecture of the proposed network, including all encoder, decoder, and latent-to-blendshape layers with their input and output dimensions, is summarized in Table 1. Here, v is the number of vertices, *feature_dim* is the per-vertex input feature size, *latent_dim* is the VAE latent size, and *bs_weight_dim* is the number of blendshape coefficients.

4.2. Hyper-parameter

In this section, we present the training results of the facial expression retargeting network. For evaluating the training, the Mean Square Error (MSE) between the ground truth and the inferred blendshape coefficients, as well as the MSE between vertices on the reconstructed 3D mesh, are used. The MSE is as per Equation (6). Here, y_i corresponds to the data inferred by the network, and t_i is the ground truth.

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - t_i)^2 \tag{6}$$

The network underwent end-to-end training. We conducted experiments by changing the loss weight and learning rate to find the optimal hyper-parameters. Through this, we aimed to find the best loss weight and learning rate. The results of the experiments are summarized in Table 2. All experiments were conducted up to 100 epochs. The learning rate experiments were performed for 10^{-3} and 10^{-4}, and the loss weight experiments were conducted for the feature reconstruction loss and retargeting loss at ratios of 10:1, 5:1, 1:1, 1:5, and 1:10. Regardless of the initial learning rate, both feature reconstruction loss and retargeting loss showed good results when the ratio was 1:1. Additionally, the prediction results of blendshape coefficients for the test dataset were best when the loss ratio was 1:1 at a learning rate of 10^{-3}, and at 10^{-4}, the best results were observed at a ratio of 5:1.

As a result of the experiments, the most optimal hyper-parameters were as shown in Table 3. Due to memory issues during training, the batch size was set to 2. Here, with the Adam optimizer's β_1 set to 0.9 and β_2 to 0.999 [33], the optimal initial learning rate was found to be 10^{-4}, and the ratio of feature reconstruction loss to retargeting loss was 5:1. At this point, the learning rate decay was set to decrease by 60% every 10 epochs.

4.3. Training results

The training results according to the retargeting loss function composition during decoder training are as follows, as shown in Table 4. When constructing the retargeting loss using only blendshape coefficients, an error of approximately 1.18×10^{-2} was observed. Training using only the vertex loss reconstructed from blendshape coefficients showed an error of about 7.13×10^{-5}. In contrast, using the proposed blend shape mask to construct the loss resulted in a significantly lower error of approximately 1.39×10^{-9} and visually provided the most stable results.

Table 1. The detailed dimension of the proposed network.

#	Module	Layer	Input dim.	Output dim.
1–16	Encoder	Gconv	$v \times$ *feature_dim*	$v \times 256$
		Gconv	$v \times 256$	$v \times 128$
		Gconv	$v \times 128$	$v \times 64$
		Gconv	$v \times 64$	$v \times 32$
		Gconv	$v \times 32$	$v \times 16$
		Gconv	$v \times 16$	$v \times 8$
		Gconv	$v \times 8$	$v \times 4$
		Gconv	$v \times 4$	$v \times 2$
		Gconv	$v \times 2$	$v \times 1$
		tanh	$v \times 1$	$v \times 1$
		Dense	2048	2048
		LeakyReLU	2048	2048
		Dense	2048	*latent_dim*
		Dense	*latent_dim*	2048
		LeakyReLU	2048	2048
		Dense	2048	$v \times 1$
17–26	Decoder	Gconv	$v \times 1$	$v \times 2$
		Gconv	$v \times 2$	$v \times 4$
		Gconv	$v \times 4$	$v \times 8$
		Gconv	$v \times 8$	$v \times 16$
		Gconv	$v \times 16$	$v \times 32$
		Gconv	$v \times 32$	$v \times 64$
		Gconv	$v \times 64$	$v \times 128$
		Gconv	$v \times 128$	$v \times 256$
		Gconv	$v \times 256$	$v \times$ *feature_dim*
		tanh	$v \times$ *feature_dim*	$v \times$ *feature_dim*
27–34	Latent → Blendshape	Dense	*latent_dim*	300
		ReLU	300	300
		Dense	300	300
		ReLU	300	300
		Dense	300	300
		ReLU	300	300
		Dense	300	*bs_weight_dim*
		tanh	*bs_weight_dim*	*bs_weight_dim*

Table 2. Experiments for finding proper hyper-parameter.

Loss Ratio	Initial Learning Rate	Feature Recon Loss	Retargeting Loss	Blendshape Coefficient MSE	Best Epoch
10:1	10^{-3}	0.0233	0.0138	0.9679	2
5:1		0.0401	0.0048	0.9963	100
1:1		0.0185	4.21×10^{-7}	0.0200	100
1:5		0.6799	0.0079	0.9755	100
1:10		0.4855	4.86×10^{-7}	0.0205	35
10:1	10^{-4}	0.0083	4.42×10^{-9}	0.0901	100
5:1		0.0091	1.42×10^{-9}	0.0699	100
1:1		0.0092	1.71×10^{-8}	0.1152	100
1:5		0.0128	0.0043	0.933	99
1:10		0.0124	0.0043	0.9351	96

Table 3. Optimal hyper-parameter.

Hyper-parameter	Value
Batch size	2
Optimization technique	Adam ($\beta_1 = 0.9, \beta_2 = 0.999$)
Initial learning rate	10^{-4}
Loss weight	5:1

Figure 6 shows the training graphs when the ratio of feature reconstruction loss to retargeting loss is 1:1, varying with the learning rate. When the initial learning rate is 10^{-3}, the retargeting loss drops to 10^{-7} as shown in Figure 6(a), whereas with an initial learning rate of 10^{-4}, it drops to 10^{-9} as shown in Figure 6(b).

Table 4. Training results according to the loss function on the test set of the AI-Hub Korean speech dataset [29].

Retargeting loss function	Vertex MSE
l_c	1.18×10^{-2}
l_v	7.13×10^{-5}
$l_c M_b l_v$	1.39×10^{-9}

Figure 7 shows the training graphs for weight loss when the initial learning rate is 10^{-4}. The simultaneous convergence of the feature reconstruction loss and the retargeting loss in the decoder showed good performance when evaluating the blendshape coefficient values for the test dataset.

Figure 8 shows the retargeting network results for the input mesh Figure 8(a), and the results of animations using different blendshapes in Figure 8(b), Figure 8(c,d). In Figure 8(b), blendshapes downloaded from the internet were animated. In Figure 8(c), blendshapes were custom-made for the FLAME model and then animated. In Figure 8(d), the result is from animating blendshapes crafted by an expert for one model. In contrast to Figure 8, which illustrated subtle lip shape variations, Figure 9 presents the results of experiments conducted

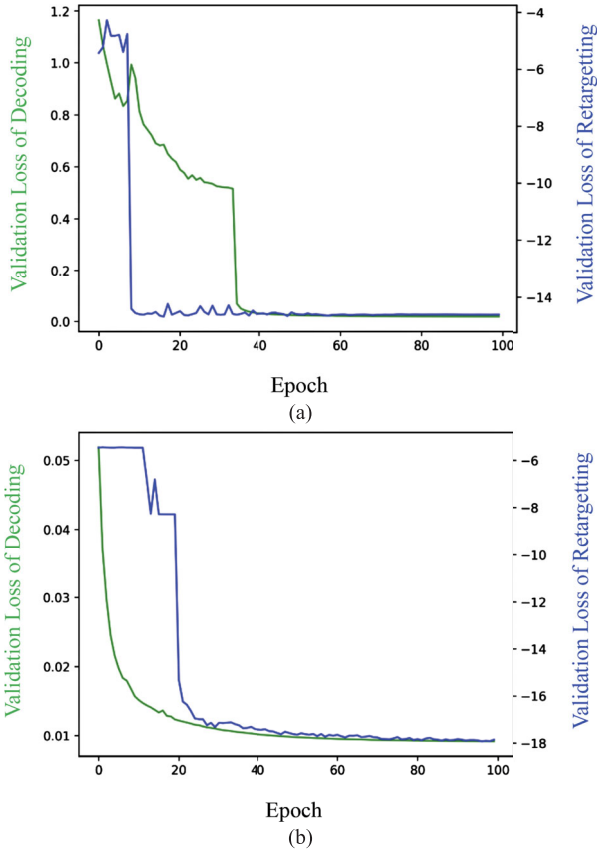

Figure 6. Training graphs with the loss weight is 1:1 and the initial learning rate of (a) 10^{-3}, (b) 10^{-4}. Two line graphs showing validation loss over training epochs for decoding and retargeting tasks. In both (a) and (b), the green line represents decoding loss (left *y*-axis) and the blue line represents retargeting loss (right *y*-axis). Graph (a) shows higher initial loss values that drop significantly around epoch 35, while graph (b) presents much lower initial losses with smooth and gradual convergence toward zero over 100 epochs.

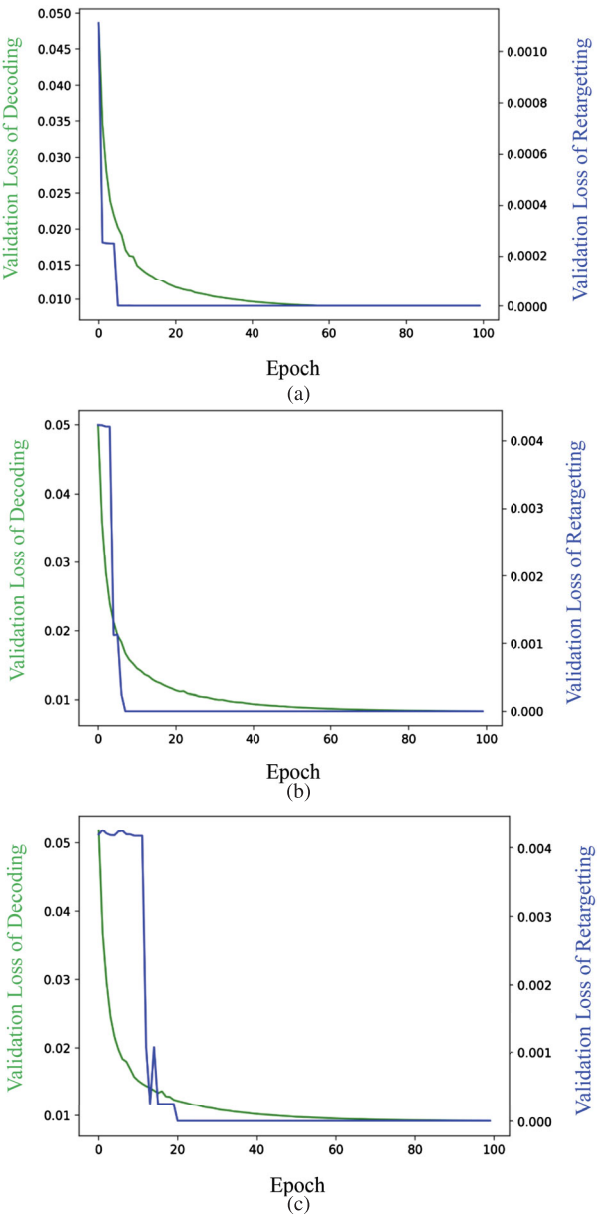

Figure 7. Training result of the proposed networks with the initial learning rate is 10^{-4} and the loss weight of (a) 10:1, (b) 5:1, (c) 1:1 during training. Three line graphs (a–c) showing the validation loss curves during training of a proposed network, each with different loss weight ratios for decoding to retargeting, the green line indicates decoding loss (left *y*-axis), and the blue line shows retargeting loss (right *y*-axis), both decreasing steadily over 100 epochs. The graphs demonstrate how different weight ratios affect convergence behavior.

using vocal inputs capable of generating pronounced facial expressions to examine changes in overall facial structure. Figure 9(a) utilizes the Arkit facial mesh, Figure 9(b) employs the FLAME mesh, and Figure 9(c) features the mesh we developed. The characteristics of facial deformation vary slightly depending on the properties of the 52 blendshape templates used for each mesh.

4.4. Quantitative evaluation

The purpose of the quantitative evaluation is to verify whether the proposed method achieves higher accuracy and stability than existing research methods. To this end, we compared our method with Sparse2Dense (S2D) and Neural Face Rigging (NFR), which are recent state-of-the-art learning-based retargeting techniques. Both baselines were selected because they represent different approaches to learning mesh deformations and blendshape rigs, thus providing meaningful points of reference for validating the effectiveness of our design.

The comparison was conducted using a subset of the CoMA dataset that emphasizes expressive facial movements such as wide jaw opening and lip articulation [9]. This subset was chosen because sequences in the standard test split often contain subtle expressions that do not

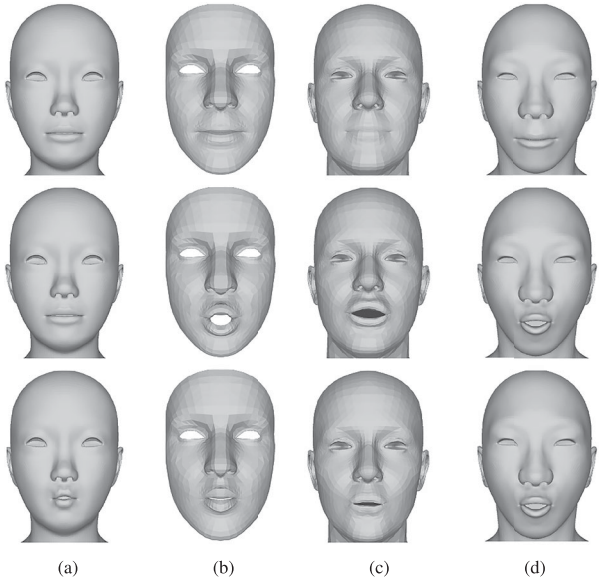

Figure 8. Facial expression retargeting test results for (a) the input facial mesh, (b) blendshape model, (c) FLAME set with 52 blendshapes, (d) blendshape model crafted by experts. A 3 × 4 grid of grayscale 3D face mesh images comparing facial expression retargeting results. Each row shows a different target facial expression.

sufficiently reveal differences among competing methods, whereas the selected subset provides a more rigorous evaluation of dynamic retargeting performance.

Table 5 presents the mean and standard deviation of vertex errors (in mm) between the ground-truth mesh and the retargeted mesh for each method.

Our method achieves the lowest error among the three approaches, with a mean error of 2.83 mm and a standard deviation of 1.79 mm. These results confirm that our method captures fine-grained facial deformations more accurately and consistently than the baseline methods, validating its stability and precision in 3D retargeting tasks.

4.5. Qualitative evaluation

While the quantitative evaluation demonstrates accuracy improvements against academic baselines, the qualitative evaluation aims to assess whether our method is suitable for real-time production pipelines. For this purpose, we compared our method with Apple's Live Link Face [34] and Google's MediaPipe [35], two widely used industry systems that represent practical standards for real-time facial motion capture and retargeting. This design ensures that our evaluation covers both research-level benchmarks and real-world deployment environments.

Figure 10 compares the similarity of blendshape coefficient estimation across the three methods. Our method

Figure 9. Facial expression retargeting test results for (a) Arkit facial mesh, (b) FLAME set with 52 blendshapes, (c) blendshape model crafted by experts. A 6 × 3 grid of shaded 3D face models showing various facial expressions. Each row represents a different expression (e.g. smiling, frowning, shouting, puckering lips).

produces coefficients that align more closely with Apple's Live Link Face, whereas Google's MediaPipe sometimes fails to distinguish shape and expression correctly (for example, inflating the cheeks or puckering lips in neutral expressions). Apple estimates neutral expressions accurately, and our results align closely with Apple's outputs,

Table 5. Quantitative comparison of retargeting accuracy on a subset of the CoMA dataset (lower is better). Mean and standard deviation of vertex error (in mm) between the ground-truth mesh and the retargeted mesh generated by S2D, NFR, and our method.

Method	Mean (mm)	Std deviation
S2D (Sparse2Dense)	3.332697	1.984873
NFR (Neural Face Rigging)	3.32212	2.056953
Ours	2.830063	1.785737

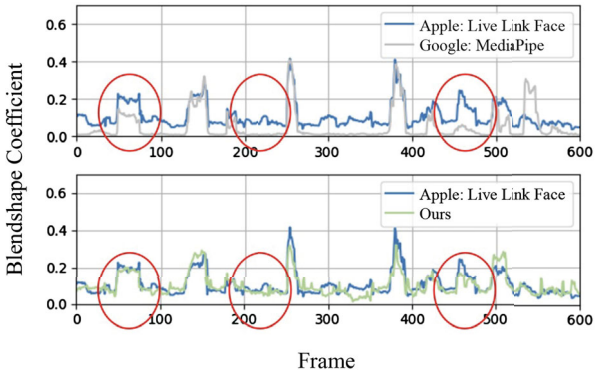

Figure 10. Comparison of the jawopen blendshape coefficients per frame. Blue represents Apple, gray represents Google, and green represents ours. The top graph compares Apple vs. Google, showing notable differences in circled segments. The bottom graph compares Apple vs. ours, showing closer alignment and smoother behavior.

confirming that our approach is not only accurate but also practically reliable. The red-marked regions highlight cases where our method better matches Apple's results than MediaPipe, suggesting improved robustness in real-world applications.

We further evaluated adaptability by applying our network to different blendshape templates. Figure 11 presents retargeting results for four settings: (a) input mesh, (b) S2D, (c) NFR, and (d) our method. Despite template variations, our approach consistently produces natural and accurate expression retargeting, particularly in the lips and jaw regions, which are critical for realism. Together, these comparisons demonstrate that our method not only surpasses academic baselines but also achieves the robustness and compatibility required for industry-grade deployment.

4.6. Runtime and complexity analysis

For real-world applications, not only accuracy but also runtime performance is an essential requirement. To evaluate the computational efficiency of our method, we compared the average runtime per frame and frames per second (FPS) against two baseline methods, Sparse2Dense (S2D) and Neural Face Rigging (NFR). All experiments were conducted on an NVIDIA RTX 4090 GPU under identical conditions for fairness. For NFR, the reported values are based on public implementation benchmarks and are presented as estimated ranges.As shown in Table 6, our method achieves real-time performance, processing at 66 FPS with an average of 15 ms per frame. This is significantly faster than S2D (16–20 FPS) and comparable or superior to NFR (30–60 FPS). These results confirm that the proposed framework is not only accurate but also efficient enough for deployment in interactive and production environments where low latency is critical.

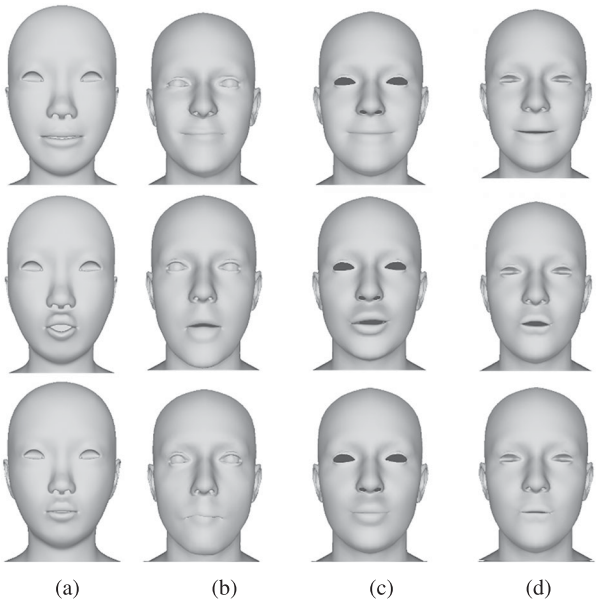

Figure 11. Facial retargeting test results: (a) input mesh, (b) S2D, (c) NFR, (d) our method. Each row shows a different facial expression, and each column compares a method. Our approach yields more accurate lip/jaw motion across different templates.

Table 6. Runtime comparison of different methods. FPS (higher is better) and time per frame (ms, lower is better) for S2D, NFR, and our method.

Method	FPS	Time per frame (ms)
S2D (Sparse2Dense)	16–20	50–60
NFR (Neural Face Rigging)	30–60 (estimated)	17–33 (estimated)
Ours	**66**	**15**

5. Conclusion

In this paper, we propose a method for transferring human 3D facial expressions to avatars. We extract expression features from FLAME facial meshes, convert them into blendshape coefficients, and demonstrate that this network allows for easy avatar animation without the need for markers. The advantage of converting extracted expression features into blendshape coefficients is that it eliminates the need for individual training for

each avatar. Additionally, when compared to Apple and Google, which estimate blendshape coefficients in a similar manner, the results of our paper are comparable to the superior results of Apple. However, there is a limitation: since blendshapes of avatars can be modeled differently by modelers in terms of the extent of mouth opening or smiling, avatars modeled with excessive or minimal movements might show inappropriate movements. Therefore, our next research goal is retargeting that takes into account the modeling of avatars.

Acknowledgments

This research was supported by the MSIT, Korea, under the ITRC support program (IITP-2022-RS-2022-00156225) supervised by the IITP. The present research has been conducted by the Excellent researcher support project of Kwangwoon University in 2024.

Author contributions

CRediT: **Sol Lee:** Conceptualization, Software, Writing – original draft; **Hak-Bum Lee:** Data curation, Formal analysis; **Ye-Won Jang:** Resources, Software, Validation; **Jung-Woo Kim:** Validation, Visualization; **Young-Ho Seo:** Conceptualization, Project administration, Supervision, Writing – review & editing

Data availability statement

This study utilized four publicly available datasets. The FLAME dataset is accessible at https://flame.is.tue.mpg.de/. The COMA dataset can be found at https://coma.is.tue.mpg.de/. The ARKit Face dataset is available at https://arkit-face-blendshapes.com/. Additionally, the AI-Hub speech-based 3D talking face dataset can be accessed through the AI-Hub repository at https://www.aihub.or.kr/aihubdata/data/view.do?dataSetSn=71683. All datasets are available online through their respective repositories, and no new data were created in this study.

Author contributions

CRediT: **Sol Lee**: Conceptualization, Software, Methodology, Writing; **Hak-Bum Lee**: Software, Methodology; **Ye-Won Jang**: Investigation; **Jungwoo Kim**: Analysis and interpretation of the data; **Young-Ho Seo**: Reviewing and editing, Supervision, Funding acquisition.

Disclosure statement

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Young-Ho Seo reports that financial support was provided by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT).

Funding

This work has supported by the National IT Industry Promotion Agency (NIPA) of Korea grant funded by the Korea government (MSIT). Grant No. RQT-25-033536, for development of [Delivery of AI solutions for SoC IP design of on-device AI NPU].

ORCID

Sol Lee http://orcid.org/0000-0002-2154-1290
Hak-Bum Lee http://orcid.org/0000-0003-0721-4944
Ye-Won Jang http://orcid.org/0000-0002-5244-5074
Jung-Woo Kim http://orcid.org/0009-0003-0913-0709
Young-Ho Seo http://orcid.org/0000-0003-1046-395X

References

[1] Choi B, Eom H, Mouscadet B, et al. Animatomy: an animator-centric, anatomically inspired system for 3D facial modeling, animation and transfer. In: Jung SK, Lee J, Bargteil AW, editors. SIGGRAPH Asia. ACM; 2022. p. 16:1–16:9.
[2] Li H, Weise T, Pauly M. Example-based facial rigging. ACM Trans Graph. Jul 2010;29(4). Article 32.
[3] Lewis JP, Anjyo K, Rhee T, et al. Practice and theory of blendshape facial models. In: Lefebvre S, Spagnuolo M, editors. Eurographics 2014 – State of the Art Reports. The Eurographics Association; 2014.
[4] Rackovic S, Soares C, Jakovetic D, et al. Clustering of the blendshape facial model. In: 2021 29th European Signal Processing Conference (EUSIPCO). Dublin, Ireland: IEEE; 2021. p. 1556–1560.
[5] Sumner R, Popović J. Deformation transfer for triangle meshe. ACM Trans Graph. Aug 2004;23:399–405.
[6] Hinton GE, Salakhutdinov RR. Reducing the dimensionality of data with neural networks. Science. 2006;313 (5786):504–507.
[7] Kingma DP, Welling M. 2022. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.
[8] Kipf TN, Welling M. 2017. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*.
[9] Ranjan A, Bolkart T, Sanyal S, et al. Generating 3D faces using convolutional mesh autoencoders. In: FerrariV, Hebert M, Sminchisescu C, et al. Computer Vision – ECCV 2018. ECCV 2018.
[10] Kipf TN, Welling M. Variational graph auto-encoders; 2016.
[11] Zhang J, Chen K, Zheng J Facial expression retargeting from human to avatar made easy. IEEE Trans Vis Comput Graph. 2022;28(2):1274–1287.
[12] Sharp N, Attaiki S, Crane K, et al. Diffusionnet: discretization agnostic learning on surfaces. ACM Trans Graph. Mar 2022;41(3):Article 27. Doi:10.1145/3507905
[13] Qin D, Saito J, Aigerman N, et al. Neural face rigging for animating and retargeting facial meshes in the wild. In: ACM SIGGRAPH 2023 Conference Proceedings (SIGGRAPH '23). New York (NY): Association for Computing Machinery; 2023. Article 68.
[14] Qin D, Saito J, Aigerman N, et al. Neural face rigging for animating and retargeting facial meshes in the wild. In: SIGGRAPH 2023 Conference Proceedings. Association for Computing Machinery; 2023.
[15] Wang T, Hu X, Liu Z, et al. Sparse2dense: learning to densify 3d features for 3d object detection. In: Advances

in Neural Information Processing Systems, vol. 35. Neural Information Processing Systems Foundation; 2022. p. 38533–38545.
[16] Otberdout N, Ferrari C, Daoudi M, et al. Generating multiple 4d expression transitions by learning face landmark trajectories. IEEE Trans Affect Comput. 2024;15(2):566–578.
[17] Otberdout N, Ferrari C, Daoudi M, et al. Sparse to dense dynamic 3d facial expression generation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). IEEE; 2022. p. 20385–20394.
[18] Dou Y, Mukai T. Facial animation retargeting by unsupervised learning of graph convolutional networks. In: 2024 Nicograph International (NicoInt). IEEE, 2024. p. 69–75.
[19] Chandran P, Zoss G, Gross M, et al. Facial animation with disentangled identity and motion using transformers. Comput Graph Forum. 2022;41(8):267–277.
[20] Blanz V, Vetter T. A morphable model for the synthesis of 3d faces. In: Proceedings of the 26th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH). ACM Press/Addison-Wesley Publishing Co., 1999; p. 187–194.
[21] Han JH, Kim JI, Kim H, et al. Generate individually optimized blendshapes. In: IEEE Transactions on Big Data and Smart Computing. 2021. p. 114–120.
[22] ARKit Face Blendshapes. Arkit face blendshapes reference. https://arkit-face-blendshapes.com/, 2024. [accessed 2024 September 16].
[23] Kim S, Jung S, Seo K, et al. Deep learning-based unsupervised human facial retargeting. Comput Graph Forum. 2021;40(7):45–55.
[24] Jiang Z-H, Wu Q, Chen K. 2019. Disentangled Representation Learning for 3D Face Shape. *arXiv preprint* arXiv:1902.09887.
[25] Gao L, Lai Y-K, Yang J, et al. Sparse data driven mesh deformation. IEEE Trans Vis Comput Graph. 2017;27:2085–2100.
[26] Z Zhang. Iterative closest point (ICP). In: Ikeuchi K, editor. Computer Vision. Cham: Springer; 2021. p. 179–193.
[27] Tang L, Zhang Y. Kullback–Leibler divergence metric learning for data distributions. IEEE Trans Pattern Anal Mach Intell. 2023;52(4):2047–2058.
[28] Zhang C-L, Luo J-H, Wei X-S, et al. In defense of fully connected layers in visual representation transfer. Neural Comput Appl. 2023.
[29] Goyang City Hall, Goyang Industry Promotion Agency, Kwangwoon University, iWeb Co., Ltd., Omotion Co., Ltd., MBC C&I Co., Ltd., and Insighter Co., Ltd. 3d speech-driven facial animation dataset. https://www.aihub.or.kr/aihubdata/data/view.do?dataSetSn=71683, 2024. Provided by AI Hub. Developed in 2023 and released in October 2024.
[30] Choi Y, Lee I, Cha S, et al. Deep-learning-based facial retargeting using local patches. Comput Graph Forum. 2025;44(1):e15263.
[31] Ahmad M, Zhang J. On the variants of relu activation function in deep learning models. Neural Comput Appl. 2022;34:1051–1070.
[32] Nazerian H, Shirazi A, Hezarkhani A. Some modified activation functions of hyperbolic tangent (tanh) activation function for artificial neural networks. Neural Comput Appl. 2022:1–14.
[33] Ahmad M, Ali T. A modified Adam algorithm for deep neural network optimization. Neural Comput Appl. 2021;33:1045–1060.
[34] Epic Games. Unreal engine facial capture with live link. https://dev.epicgames.com/community/learning/tutorials/lEYe/unreal-engine-facial-capture-with-live-link, 2024. [accessed 2024 septmeber 2016].
[35] Google AI. Mediapipe framework for on-device ai. *Google AI Developer Platform*, 2024.